Panorama Health Check

XML API reference: every call the tool makes, how it is sent, what comes back and what is checked

14	2	1 / sec	0
allowlisted read-only <show> commands	request types used (op, log)	one request at a time, 1 second apart	write operations possible

Document	API reference for the read-only Panorama health-check collector
Version / date	1.1, 3 November 2026
API	PAN-OS XML API on the lab Panorama (https://<panorama>/api/)
Scope	The default health check: Panorama status, firewall inventory, sync, jobs, log collectors, logs and the opt-in per-firewall checks
Access needed	An API key whose admin role has XML API **Operational Requests** and **Log**
Code	`pano_healthcheck.py` (Python standard library only), tests in `tests/test_pano_healthcheck.py`

Contents

1. Overview . . . 3

1.1 Architecture . . . 3

2. API conventions . . . 4

2.1 Endpoint and transport . . . 4

2.2 Authentication . . . 4

2.3 Request parameters . . . 4

2.4 Response format . . . 4

2.5 Error codes . . . 5

2.6 Throttling and retries . . . 5

2.7 Request lifecycle . . . 6

3. Endpoint reference . . . 7

Summary of calls in a default run . . . 7

3.1 Panorama status . . . 8

3.2 Firewalls and sync . . . 13

3.3 Jobs and log collectors . . . 16

3.4 Log searches . . . 18

3.5 Per-firewall live checks (opt-in) . . . 21

4. Health findings reference . . . 27

5. Report outputs . . . 28

6. Configuration reference (.env) . . . 29

Command-line options . . . 29

7. Safety controls . . . 30

8. Python code structure . . . 31

How a call is made in code . . . 31

The allowlist (shortened) . . . 31

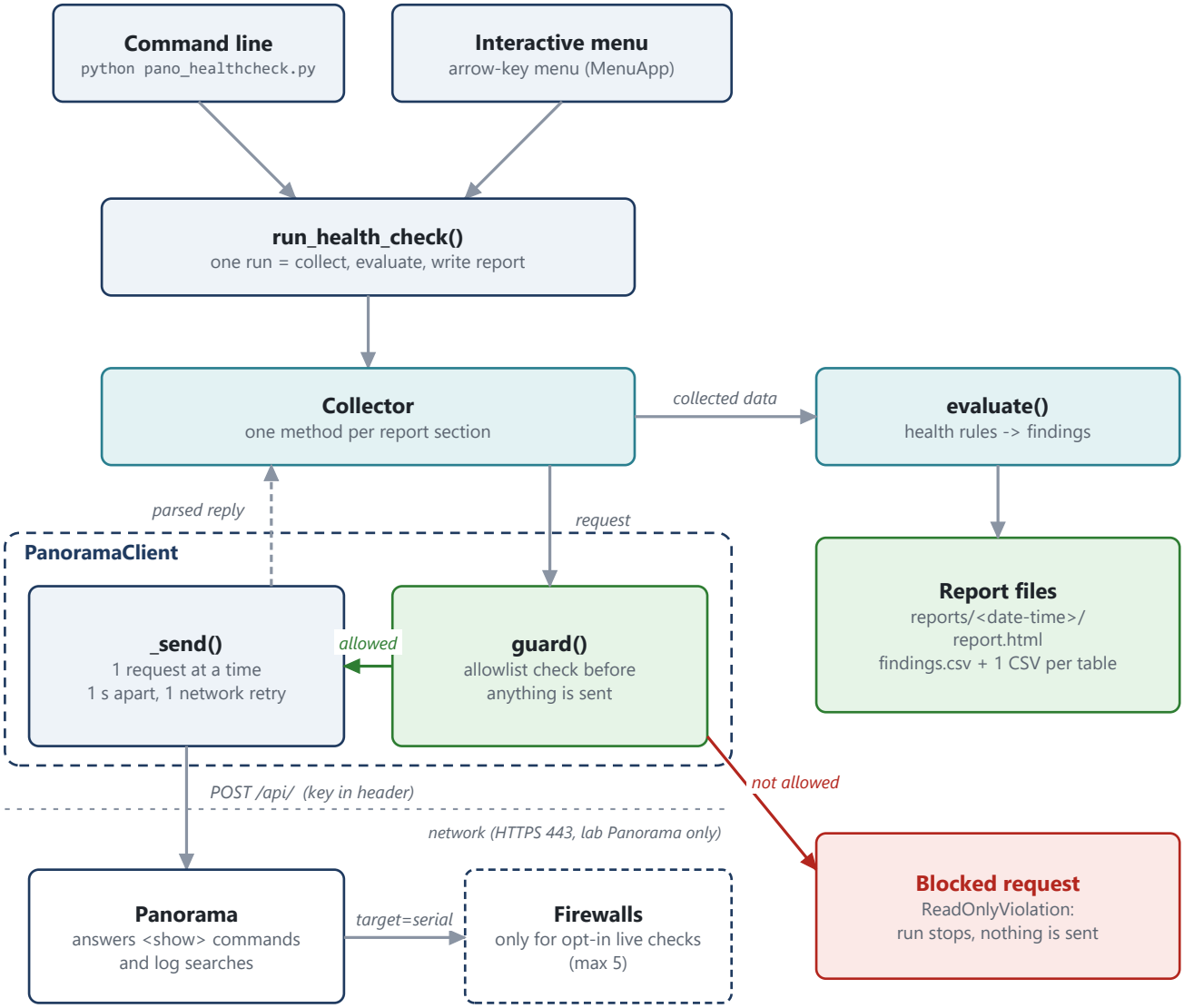
Appendix A. Command allowlist . . . 32

1. Overview

The tool connects to the lab Panorama and asks it a fixed list of read-only questions through the PAN-OS XML API. It applies health rules to the answers and writes an HTML report and CSV files. It **cannot change anything** on Panorama or on the firewalls: every request is checked against an allowlist before it is sent, and anything not on the list is blocked.

Most data comes from a handful of Panorama-wide calls (for example, one call returns the whole firewall inventory). The tool only contacts individual firewalls, through Panorama, when the opt-in per-firewall checks are switched on.

1.1 Architecture



- **Only one part of the code sends traffic.** Everything goes through PanoramaClient, so the safety check cannot be skipped.
- **Firewalls are reached through Panorama** (target=serial). The tool never connects to a firewall directly.
- **A failed section does not stop the run.** The error is written to collection_errors.csv and the other sections are still collected.

2. API conventions

These rules apply to every call in section 3.

2.1 Endpoint and transport

Item	Value
URL	`https://<PANORAMA_HOST>:<PANORAMA_PORT>/api/` (port 443 by default)
Method	HTTP POST, with the parameters in a form-encoded body (HTTP_METHOD=POST)
Content type	`application/x-www-form-urlencoded`
Response	XML, always wrapped in a <response> element
TLS	HTTPS only. Certificate checking is controlled by VERIFY_TLS and CA_BUNDLE; the lab default is off, and the tool prints a warning when it is.
Proxy	Bypassed by default (BYPASS_PROXY=true), so traffic goes straight to Panorama
Timeout	60 seconds per request (TIMEOUT_SEC)
Allowed host	Only hosts listed in ALLOWED_HOSTS in the .env file. Anything else is refused before a connection is made.

2.2 Authentication

Every request carries the XML API key in the `X-PAN-KEY` HTTP header (KEY_IN_HEADER=true). The key is never put in the URL, so it does not appear in proxy logs or web server access logs. The tool refuses to start if it is configured to send the key in a GET URL. The key comes from `PANORAMA_XML_API_KEY` in .env (`PANORAMA_API_KEY` is the fallback) and is never written to reports, CSVs or screen output.

Panorama checks the key's **admin role**. Each XML API request type is a separate permission on the role's XML API tab:

Role permission	Request type	Used for
Operational Requests	`type=op`	All <show> commands: sections 3.1, 3.2, 3.3 and 3.5
Log	`type=log`	Log searches: section 3.4

2.3 Request parameters

These are the only parameters the tool ever sends. guard() rejects any other parameter name, and any value that is not allowed, before the request leaves the machine.

Parameter	Request type	Allowed values
`type`	all	`op` or `log`
`cmd`	op	One of the 14 exact XML strings in OP_COMMANDS (appendix A). Never built from user input.
`target`	op	A firewall serial number (letters, digits, - and _, up to 64 characters). Only allowed on the 8 commands that may be forwarded to a firewall.
`log-type`	log	`system` or `config`
`nlogs`	log	A number from 1 to 5000 (the tool sends MAX_LOGS = 100)
`dir`	log	`backward` (newest first)
`query`	log	A filter on severity and receive_time, built from the settings
`action`	log	`get` only, to fetch a finished search
`job-id`	log	Digits only, from the reply to a log search

2.4 Response format

A successful reply has `status="success"`; the data is inside `<result>`. The tool parses it with Python's built-in XML parser and flattens it into simple field/value pairs.

Success

```
<response status="success">
  <result>
    ... command-specific data ...
  </result>
</response>
```

An error reply has `status="error"`, a numeric `code` and a message:

Error

```
<response status="error" code="403">
  <result><msg>Type [config] not authorized for user role.</msg></result>
</response>
```

2.5 Error codes

The tool translates Panorama's codes into plain English and, where it can, adds what to change. The error is saved in collection_errors.csv and shown in the report; the rest of the run continues.

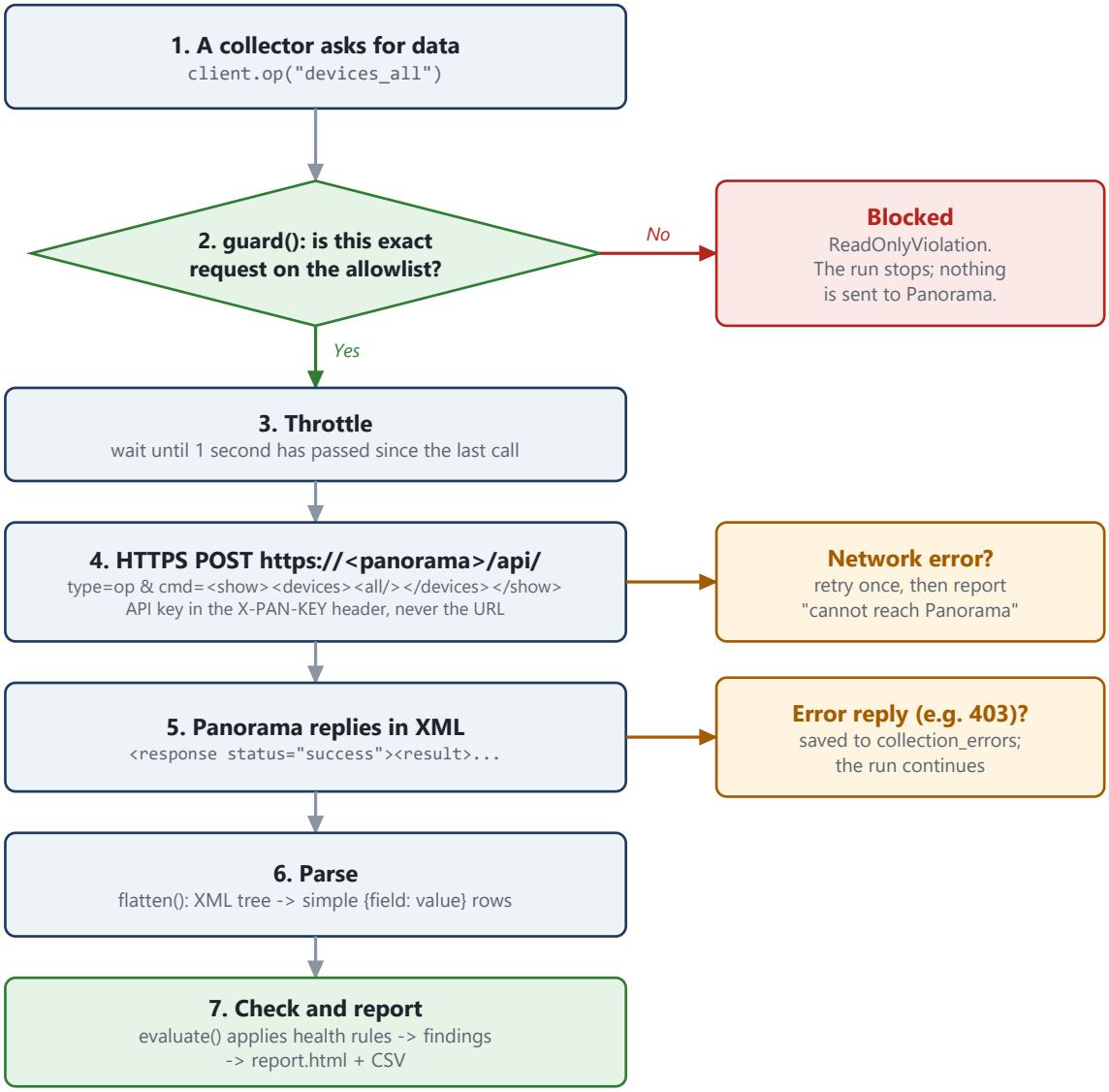
Code	Meaning	What the tool does / suggests
`403 / 16`	Forbidden: the key is valid but its role lacks this request type	Names the missing XML API permission (Panorama > Admin Roles > role > XML API) and says to commit on Panorama
`401`	Unauthorized: invalid API key	Check the key in .env
`22`	Session timed out	Generate a new API key
`1 / 17 / 18`	Unknown, invalid or malformed command	The command may not exist on this PAN-OS version. Per-firewall checks skip just that command.
`6`	Bad xpath	Recorded as an error
`7 / 13`	Object not present / not found	Recorded as an error
`2-5, 11, 21`	Internal error on Panorama	Recorded as an error
`not XML`	A proxy or the wrong host answered	Reports 'response was not XML (proxy or wrong host?)'
`network`	Connection refused, timeout, DNS failure	Waits, retries **once**, then reports 'cannot reach <host>'

2.6 Throttling and retries

- Requests are sent **one at a time**; there are no threads or parallel requests.
- There are at least **REQUEST_DELAY_SEC (1 s)** between the end of one request and the start of the next.
- A network failure is retried **once**, after at least 2 seconds. Error replies from Panorama are not retried.
- Log searches are bounded: at most 100 entries each, only the last 24 hours, and at most ~55 seconds of polling.
- Per-firewall checks are opt-in and limited to 5 firewalls (MAX_TARGETED_FIREWALLS). If a firewall is unreachable or the request is denied, its remaining commands are skipped.

2.7 Request lifecycle

Every call follows the same seven steps. The example is the firewall inventory call.



3. Endpoint reference

Each card below documents one call: the parameters, an example request and response, which response fields the tool reads, and which health findings they can raise. On the wire the request body is URL-encoded; the examples show it decoded, one parameter per line, for readability.

Summary of calls in a default run

Call	Type	Section	Purpose	Calls
`system_info`	op	panorama	Panorama identity and PAN-OS version	1
`ha_state`	op	panorama	Panorama HA state	1
`system_resources`	op	panorama	Panorama CPU and memory	1
`disk_space`	op	panorama	Panorama disk usage	1
`ntp`	op	panorama	Panorama clock synchronisation	1
`devices_all`	op	devices	All managed firewalls: state, versions, HA	1
`devicegroups`	op	sync	Device-group members and policy sync	1
`templates`	op	sync	Template members and template sync	1
`jobs_all`	op	jobs	Commit and push results	1
`log_collectors`	op	collectors	Log collector connectivity	1
`log query`	log	logs	Start system and config log searches	2
`log get`	log	logs	Fetch the search results	2+
Total			Without per-firewall checks	**14+**
`ha_all ... disk_space + target=serial`	op	targeted (opt-in)	7 live checks on each selected firewall, max 5 firewalls	7 each

A typical run with live checks on 5 firewalls makes about 14 + 35 = **49 calls**, which takes roughly a minute at one call per second. `python pano_healthcheck.py --dry-run` lists the exact calls for your environment without sending anything.

3.1 Panorama status

`system_info` Panorama system information

CLI equivalent: show system info

METHOD	REQUEST TYPE	REPORT SECTION	PERMISSION	CALLS PER RUN
POST /api/	**op**	**panorama**	**Operational Requests**	**1**

Identifies the Panorama: hostname, model, serial, PAN-OS version and the content versions it has. Its PAN-OS version is compared with every firewall's, because Panorama must run the same or a later release than the firewalls it manages.

Parameters

Parameter	Value	Notes
`type`	op	Operational command.
`cmd`	`<show><system><info></info></system></show>`	Must match an OP_COMMANDS entry exactly, character for character.

Example request

```
POST /api/ HTTP/1.1
Host: <panorama>
X-PAN-KEY: <XML API key>
Content-Type: application/x-www-form-urlencoded

type=op&
cmd=<show><system><info></info></system></show>
```

Example response

Sample data from the test suite, trimmed.

```
<response status="success"><result><system>
  <hostname>pano-lab</hostname>
  <model>Panorama</model>
  <sw-version>11.1.4</sw-version>
  <uptime>3 days</uptime>
  ...
</system></result></response>
```

Fields read

Response field	How the tool uses it
`hostname, model, serial, uptime`	Shown in the Panorama section of the report.
`sw-version`	Compared with each firewall's sw-version.
`app-version, av-version, wildfire-version`	Part of the "newest content in use" baseline.

Findings raised

Severity	Finding raised when...
high	A firewall runs a newer PAN-OS feature release (e.g. 11.2 vs 11.1) than Panorama.
low	A firewall runs a newer maintenance release (e.g. 11.1.6 vs 11.1.4) than Panorama.

`ha_state` Panorama high availability

CLI equivalent: show high-availability state

METHOD	REQUEST TYPE	REPORT SECTION	PERMISSION	CALLS PER RUN
POST /api/	**op**	**panorama**	**Operational Requests**	**1**

Checks whether Panorama itself is in an HA pair and, if so, whether the pair is healthy.

Parameters

Parameter	Value	Notes
`type`	`op`	Operational command.
`cmd`	`<show><high-availability><state></state></high-availability></show>`	Must match an OP_COMMANDS entry exactly, character for character.

Example request

```
POST /api/ HTTP/1.1
Host: <panorama>
X-PAN-KEY: <XML API key>
Content-Type: application/x-www-form-urlencoded

type=op&
cmd=<show><high-availability><state></state></high-availability></show>
```

Example response

Sample data from the test suite, trimmed.

```
<response status="success"><result>
  <enabled>yes</enabled>
  <group>
    <local-info><state>primary-suspended</state></local-info>
    <peer-info><conn-status>down</conn-status><state>secondary-active</state></peer-info>
    <running-sync>not synchronized</running-sync>
    <running-sync-enabled>yes</running-sync-enabled>
  </group>
</result></response>
```

Fields read

Response field	How the tool uses it
`enabled`	Checks are skipped when HA is not enabled.
`local-info/state`	Panorama's own HA state.
`peer-info/conn-status`	Connection to the HA peer.
`running-sync, running-sync-enabled`	Whether both Panoramas have the same running config.

Findings raised

Severity	Finding raised when...
high	Peer connection is not 'up'.
high	Local state is neither active nor passive (e.g. suspended).
medium	Running config is not 'synchronized' (and running-sync is enabled).

`system_resources` Panorama CPU and memory

CLI equivalent: show system resources (a snapshot of the Linux 'top' command)

METHOD	REQUEST TYPE	REPORT SECTION	PERMISSION	CALLS PER RUN
POST /api/	**op**	**panorama**	**Operational Requests**	**1**

Returns plain text from 'top'. The tool reads the CPU idle percentage and the memory totals from it.

Parameters

Parameter	Value	Notes
`type`	`op`	Operational command.
`cmd`	`<show><system><resources></resources></system></show>`	Must match an OP_COMMANDS entry exactly, character for character.

Example request

```
POST /api/ HTTP/1.1
Host: <panorama>
X-PAN-KEY: <XML API key>
Content-Type: application/x-www-form-urlencoded

type=op&
cmd=<show><system><resources></resources></system></show>
```

Example response

Sample data from the test suite, trimmed.

```
<response status="success"><result><![CDATA[top - 10:00:00 up 3 days
%Cpu(s):  7.5 us,  2.5 sy,  0.0 ni, 90.0 id,  0.0 wa
MiB Mem :   8000.0 total,   1000.0 free,   6000.0 used,   1000.0 buff/cache
]]></result></response>
```

Fields read

Response field	How the tool uses it
`%Cpu(s): ... id`	mgmt_cpu_pct = 100 - idle (here 10.0 %).
`Mem: total / used`	mgmt_mem_pct = used / total (here 75.0 %). Older PAN-OS 'k total' format also read.

Findings raised

Severity	Finding raised when...
high	Management CPU is 85 % or more.
high	Management memory is 90 % or more.

`disk_space` Panorama disk usage

CLI equivalent: show system disk-space (the Linux 'df -h' command)

METHOD	REQUEST TYPE	REPORT SECTION	PERMISSION	CALLS PER RUN
POST /api/	**op**	**panorama**	**Operational Requests**	**1**

Returns 'df -h' text. The tool turns each /dev filesystem line into a row.

Parameters

Parameter	Value	Notes
`type`	`op`	Operational command.
`cmd`	`<show><system><disk-space></disk-space></system></show>`	Must match an OP_COMMANDS entry exactly, character for character.

Example request

```
POST /api/ HTTP/1.1
Host: <panorama>
X-PAN-KEY: <XML API key>
Content-Type: application/x-www-form-urlencoded

type=op&
cmd=<show><system><disk-space></disk-space></system></show>
```

Example response

Sample data from the test suite, trimmed.

```
<response status="success"><result><![CDATA[Filesystem  Size  Used Avail Use% Mounted on
/dev/root   6.9G  6.3G  0.3G  95% /
/dev/sda5    16G  9.6G  5.2G  65% /opt/pancfg
/dev/sda8   200G  196G  4.0G  98% /opt/panlogs
]]></result></response>
```

Fields read

Response field	How the tool uses it
`Filesystem, Size, Used, Avail`	Copied to disk_usage.csv.
`Use%`	Compared with the disk thresholds.
`Mounted on`	/opt/panlogs is skipped: PAN-OS keeps log storage full by design and deletes the oldest logs.

Findings raised

Severity	Finding raised when...
high	A filesystem is 90 % full or more.
medium	A filesystem is 80 % full or more.

`ntp` Panorama clock (NTP)

CLI equivalent: show ntp

METHOD	REQUEST TYPE	REPORT SECTION	PERMISSION	CALLS PER RUN
POST /api/	**op**	**panorama**	**Operational Requests**	**1**

Checks that Panorama's clock is synchronised to an NTP server. Log times, job times and certificate checks all depend on a correct clock.

Parameters

Parameter	Value	Notes
`type`	op	Operational command.
`cmd`	`<show><ntp></ntp></show>`	Must match an OP_COMMANDS entry exactly, character for character.

Example request

```
POST /api/ HTTP/1.1
Host: <panorama>
X-PAN-KEY: <XML API key>
Content-Type: application/x-www-form-urlencoded

type=op&
cmd=<show><ntp></ntp></show>
```

Example response

Sample data from the test suite, trimmed.

```
<response status="success"><result>
  <synched>LOCAL</synched>
  <ntp-server-1>
    <name>192.0.2.123</name><status>rejected</status><reachable>no</reachable>
  </ntp-server-1>
</result></response>
```

Fields read

Response field	How the tool uses it
`synched`	The server the clock follows. LOCAL means no NTP server is in use.
`ntp-server-N/name, status, reachable`	Listed in the report for each configured server.

Findings raised

Severity	Finding raised when...
medium	synched is LOCAL or empty: the clock is not synchronised to an NTP server.

3.2 Firewalls and sync

`devices_all` Managed firewall inventory

CLI equivalent: show devices all

METHOD	REQUEST TYPE	REPORT SECTION	PERMISSION	CALLS PER RUN
POST /api/	**op**	**devices**	**Operational Requests**	**1**

The most important call. One Panorama-wide request returns every managed firewall with its connection state, model, versions, HA state, device certificate and policy sync status, so the tool does not need to contact each firewall.

Parameters

Parameter	Value	Notes
`type`	op	Operational command.
`cmd`	`<show><devices><all></all></devices></show>`	Must match an OP_COMMANDS entry exactly, character for character.

Example request

```
POST /api/ HTTP/1.1
Host: <panorama>
X-PAN-KEY: <XML API key>
Content-Type: application/x-www-form-urlencoded

type=op&
cmd=<show><devices><all></all></devices></show>
```

Example response

Sample data from the test suite, trimmed.

```
<response status="success"><result><devices>
  <entry name="001">
    <serial>001</serial><connected>yes</connected><hostname>fw-a</hostname>
    <ip-address>192.0.2.21</ip-address><model>PA-VM</model><sw-version>10.2.10</sw-version>
    <app-version>8800-8000</app-version><av-version>4800-5300</av-version>
    <ha><state>active</state><peer><serial>002</serial></peer></ha>
    <vsys><entry name="vsys1"><shared-policy-status>In Sync</shared-policy-status></entry></vsys>
  </entry>
  <entry name="003"><serial>003</serial><connected>no</connected><hostname>fw-c</hostname>...</entry>
</devices></result></response>
```

Fields read

Response field	How the tool uses it
`serial, hostname, ip-address, model, family`	Inventory columns.
`connected`	yes / no: is the firewall connected to Panorama?
`deactivated, unsupported-version`	Inventory flags.
`sw-version`	PAN-OS version: fleet majority and Panorama comparison.
`app-version, av-version, wildfire-version`	Content versions: compared with the newest in use.
`ha/state, ha/peer/serial`	Used to pair HA members and check the pair.
`device-cert-present, device-cert-expiry-date`	Device certificate (needed for cloud services).
`*-policy-status, *template-status, last-commit-all-*`	Policy / template sync and last push result.

Findings raised

Severity	Finding raised when...
critical	Firewall is not connected to Panorama.
critical	HA pair: both members are active (possible split brain).
high	HA state is not healthy (e.g. suspended, non-functional).
high	HA pair: no active member; or the two peers run different PAN-OS / content versions.
high	A policy or template status is 'Out of Sync', or the last commit/push failed.
medium	HA pair: the peer is not connected to Panorama.
medium	Device certificate is missing / expired, or expires within 30 days.
medium	Version is flagged by Panorama as unsupported.
medium	Antivirus content is 3 or more releases behind the newest in use.
low	Other content (apps & threats, WildFire) behind the newest in use; HA peer not managed / not reported.
info	Firewall is deactivated; PAN-OS version differs from the fleet majority.

`devicegroups` Device-group membership and policy sync

CLI equivalent: show devicegroups

METHOD	REQUEST TYPE	REPORT SECTION	PERMISSION	CALLS PER RUN
POST /api/	**op**	**sync**	**Operational Requests**	**1**

Lists every device group and its member firewalls, with each member's policy sync status. The device-group names are also cached for the interactive menu.

Parameters

Parameter	Value	Notes
`type`	`op`	Operational command.
`cmd`	`<show><devicegroups></devicegroups></show>`	Must match an OP_COMMANDS entry exactly, character for character.

Example request

```
POST /api/ HTTP/1.1
Host: <panorama>
X-PAN-KEY: <XML API key>
Content-Type: application/x-www-form-urlencoded

type=op&
cmd=<show><devicegroups></devicegroups></show>
```

Example response

Sample data from the test suite, trimmed.

```
<response status="success"><result><devicegroups>
  <entry name="DG-Branch"><devices>
    <entry name="001"><serial>001</serial><connected>yes</connected><hostname>fw-a</hostname>
      <shared-policy-status>In Sync</shared-policy-status></entry>
  </devices></entry>
</devicegroups></result></response>
```

Fields read

Response field	How the tool uses it
`entry/@name`	Device-group name.
`devices/entry: serial, hostname, connected`	Member firewall.
`any field containing 'status', 'sync' or 'last-commit'`	Combined into one status: Out of Sync, Commit failed, In Sync or unknown.

Findings raised

Severity	Finding raised when...
high	A member's status is out of sync or shows a failure.

`templates` Template membership and template sync

CLI equivalent: show templates

METHOD	REQUEST TYPE	REPORT SECTION	PERMISSION	CALLS PER RUN
POST /api/	**op**	**sync**	**Operational Requests**	**1**

Same as devicegroups, for templates (network and device settings pushed from Panorama).

Parameters

Parameter	Value	Notes
`type`	`op`	Operational command.
`cmd`	`<show><templates></templates></show>`	Must match an OP_COMMANDS entry exactly, character for character.

Example request

```
POST /api/ HTTP/1.1
Host: <panorama>
X-PAN-KEY: <XML API key>
Content-Type: application/x-www-form-urlencoded

type=op&
cmd=<show><templates></templates></show>
```

Example response

Sample data from the test suite, trimmed.

```
<response status="success"><result><templates>
  <entry name="TPL-Branch"><devices>
    <entry name="001"><serial>001</serial><connected>yes</connected><hostname>fw-a</hostname>
      <template-status>Out of Sync</template-status></entry>
  </devices></entry>
</templates></result></response>
```

Fields read

Response field	How the tool uses it
`entry/@name`	Template name.
`devices/entry: serial, hostname, template-status`	Member firewall and its template sync status.

Findings raised

Severity	Finding raised when...
high	A member's template status is out of sync or shows a failure.

Notes

- If this call fails, device-group sync is still reported; the error is recorded separately as 'sync (templates)'.

3.3 Jobs and log collectors

`jobs_all` Panorama jobs (commits and pushes)

CLI equivalent: show jobs all

METHOD	REQUEST TYPE	REPORT SECTION	PERMISSION	CALLS PER RUN
POST /api/	**op**	**jobs**	**Operational Requests**	**1**

Lists Panorama's recent jobs: commits, pushes to firewalls, content downloads and installs. The newest 50 (MAX_JOBS) are kept.

Parameters

Parameter	Value	Notes
`type`	op	Operational command.
`cmd`	`<show><jobs><all></all></jobs></show>`	Must match an OP_COMMANDS entry exactly, character for character.

Example request

```
POST /api/ HTTP/1.1
Host: <panorama>
X-PAN-KEY: <XML API key>
Content-Type: application/x-www-form-urlencoded

type=op&
cmd=<show><jobs><all></all></jobs></show>
```

Example response

Sample data from the test suite, trimmed.

```
<response status="success"><result>
  <job><id>42</id><type>CommitAll</type><status>FIN</status><result>FAIL</result>
    <tfin>2026/10/28 11:00:00</tfin>
    <devices><entry name="002"><devicename>fw-b</devicename><result>FAIL</result></entry></devices>
  </job>
</result></response>
```

Fields read

Response field	How the tool uses it
`id, type, status, result, user, tenq, tfin, progress`	Job columns (tenq = queued, tfin = finished).
`details`	Job messages, trimmed to 500 characters.
`devices/entry/result`	Per-firewall result of a push: counted as devices_total / devices_failed.

Findings raised

Severity	Finding raised when...
high	A job's result is FAIL, or at least one firewall in a push failed.

`log_collectors` Log collector connectivity

CLI equivalent: show log-collector connected

METHOD	REQUEST TYPE	REPORT SECTION	PERMISSION	CALLS PER RUN
POST /api/	**op**	**collectors**	**Operational Requests**	**1**

Lists the log collectors connected to Panorama.

Parameters

Parameter	Value	Notes
`type`	op	Operational command.
`cmd`	`<show><log-collector><connected></connected></log-collector></show>`	Must match an OP_COMMANDS entry exactly, character for character.

Example request

```
POST /api/ HTTP/1.1
Host: <panorama>
X-PAN-KEY: <XML API key>
Content-Type: application/x-www-form-urlencoded

type=op&
cmd=<show><log-collector><connected></connected></log-collector></show>
```

Example response

Sample data from the test suite, trimmed.

```
<response status="success"><result><log-collector>
  <entry name="LC1"><host-name>lc-1</host-name><connected>no</connected></entry>
</log-collector></result></response>
```

Fields read

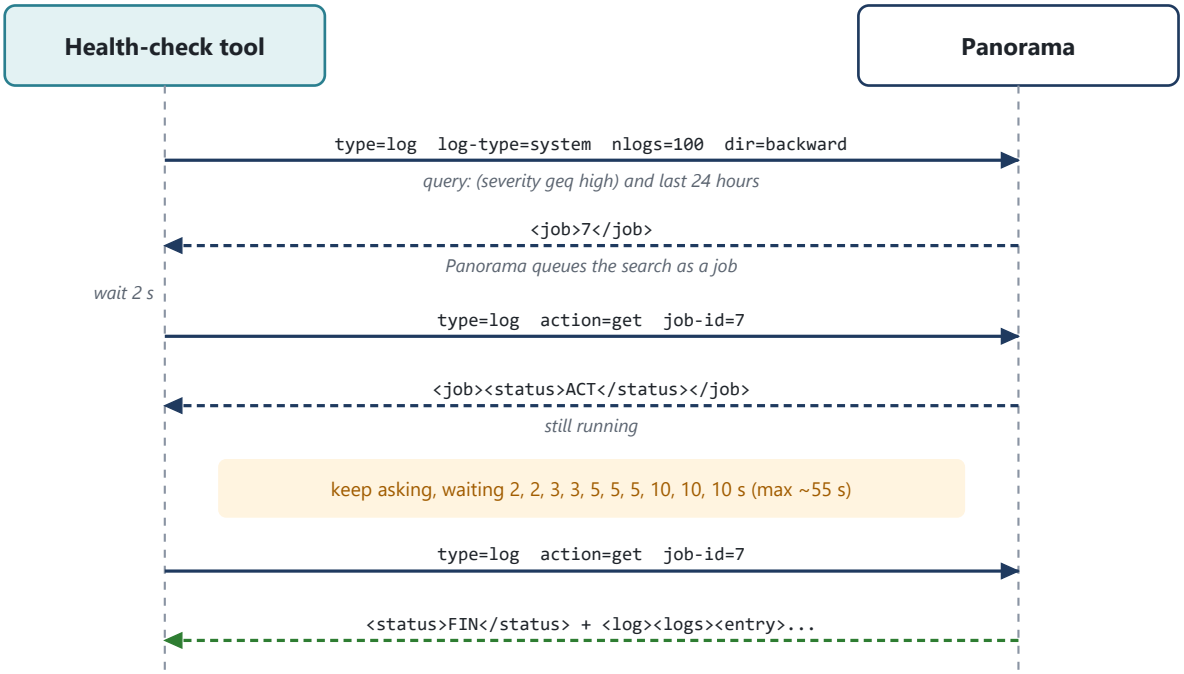
Response field	How the tool uses it
`entry/@name (serial), host-name`	Log collector identity.
`connected`	yes / no.

Findings raised

Severity	Finding raised when...
high	A log collector is not connected.

3.4 Log searches

Log searches on Panorama run in the background, so a search takes two kinds of call. The tool first submits the search ('log query') and gets a job number back. It then asks for the result ('log get'), waiting longer between each check, until Panorama says the job is finished (FIN). This happens twice per run: once for system logs and once for configuration logs.



`log query` Start a log search

Submits a search to Panorama's log database. Returns a job number, not the logs.

METHOD	REQUEST TYPE	REPORT SECTION	PERMISSION	CALLS PER RUN
POST /api/	**log**	**logs**	**Log**	**2 (system, config)**

Run twice per health check: once for system logs and once for configuration logs.

Parameters

Parameter	Value	Notes
`type`	`log`	Log request.
`log-type`	`system | config`	Only these two log types are allowed by guard().
`nlogs`	`100`	MAX_LOGS. guard() refuses anything outside 1 to 5000.
`dir`	`backward`	Newest first.
`query`	`( severity geq high ) and ( receive_time geq '2026/11/02 09:00:00' )`	System logs: high and critical in the last LOG_WINDOW_HOURS (24 h). Config logs: the time filter only.

Example request

```
POST /api/ HTTP/1.1
Host: <panorama>
X-PAN-KEY: <XML API key>
Content-Type: application/x-www-form-urlencoded

type=log&
log-type=system&
nlogs=100&
dir=backward&
query=( severity geq high ) and ( receive_time geq '2026/11/02 09:00:00' )
```

Example response

Sample data from the test suite, trimmed.

```
<response status="success"><result>
  <msg><line>query job enqueued with jobid 7</line></msg>
  <job>7</job>
</result></response>
```

Fields read

Response field	How the tool uses it
job	The job number used by 'log get'. Anything other than digits is an error.

`log get` Fetch log search results

Asks for the result of a log search job until Panorama reports it finished (FIN).

METHOD	REQUEST TYPE	REPORT SECTION	PERMISSION	CALLS PER RUN
POST /api/	**log**	**logs**	**Log**	**1 or more per search**

Polled with growing waits of 2, 2, 3, 3, 5, 5, 5, 10, 10 and 10 seconds. If the job has not finished after about 55 seconds, the tool stops asking and records an error for that log type.

Parameters

Parameter	Value	Notes
type	log	
action	get	The only log action allowed.
job-id	7	Digits only; no other parameters are allowed with action=get.

Example request

```
POST /api/ HTTP/1.1
Host: <panorama>
X-PAN-KEY: <XML API key>
Content-Type: application/x-www-form-urlencoded

type=log&
action=get&
job-id=7
```

Example response

Sample data from the test suite, trimmed.

```
<response status="success"><result>
  <job><status>FIN</status></job>
  <log><logs count="1">
    <entry logid="1"><receive_time>2026/10/28 09:00:00</receive_time>
      <severity>critical</severity><opaque>disk full</opaque><device_name>fw-a</device_name>
    </entry>
  </logs></log>
</result></response>
```

Fields read

Response field	How the tool uses it
`job/status`	ACT = still running, FIN = finished.
`System log entry: receive_time, time_generated, device_name, serial, severity, subtype, eventid, opaque`	Columns of system_logs.csv (opaque is the event description).
`Config log entry: receive_time, admin, client, cmd, path, result, device_name, serial`	Columns of config_logs.csv: who changed what, from where, and whether it succeeded.

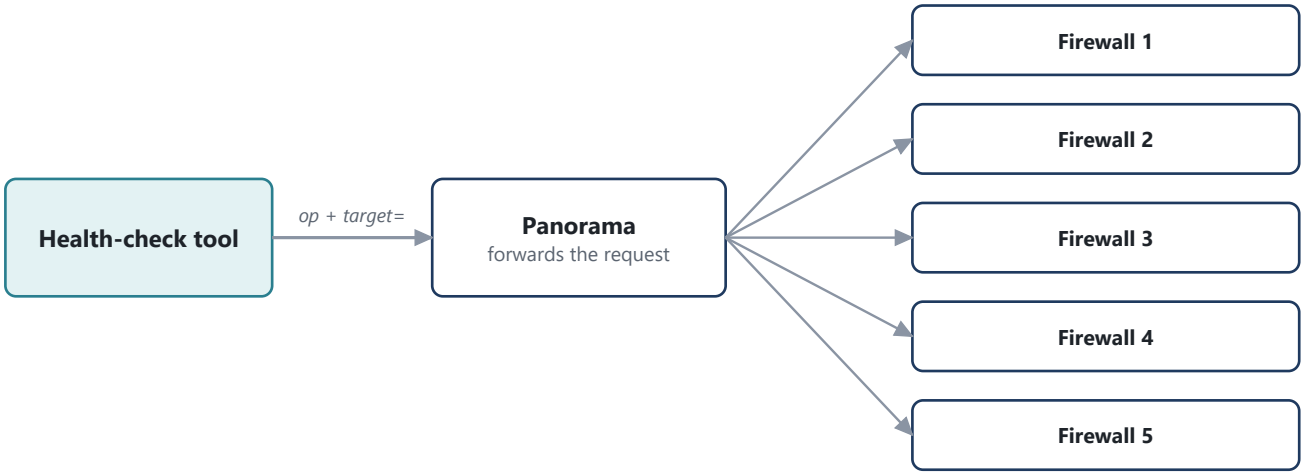
Findings raised

Severity	Finding raised when...
medium	One or more critical system log events in the log window.
info	Number of configuration changes in the log window.

3.5 Per-firewall live checks (opt-in)

Some health data only exists on the firewall itself. For these checks the tool asks Panorama to forward a show command to one firewall, by adding `target=` and the firewall's serial number. Panorama relays the command over its existing management connection and returns the firewall's answer.

This is **off by default**. It is switched on with --targeted, --firewalls, ENABLE_TARGETED=true or from the menu, and it only covers firewalls that are connected, up to 5 per run. Each selected firewall gets 7 commands, one at a time: ha_all, session_info, system_resources, dp_resources, logging_status, system_info and disk_space.



`ha_all` Firewall HA detail

CLI equivalent: show high-availability all (on the firewall)

METHOD	REQUEST TYPE	REPORT SECTION	PERMISSION	CALLS PER RUN
POST /api/	**op**	**targeted**	**Operational Requests**	**1 per firewall**

Full HA detail from the firewall itself: HA mode, both members' states, HA1/HA2 link status, and whether the pair's configuration is synchronised. Panorama's inventory does not have this detail.

Parameters

Parameter	Value	Notes
`type`	`op`	Operational command.
`cmd`	`<show><high-availability><all></all></high-availability></show>`	Must match an OP_COMMANDS entry exactly, character for character.
`target`	`<firewall serial>`	Only for per-firewall checks. Letters, digits, - and _ only.

Example request

```
POST /api/ HTTP/1.1
Host: <panorama>
X-PAN-KEY: <XML API key>
Content-Type: application/x-www-form-urlencoded

type=op&
cmd=<show><high-availability><all></all></high-availability></show>&
target=007951000123456
```

Example response

Sample data from the test suite, trimmed.

```
<response status="success"><result><enabled>yes</enabled><group>
  <mode>Active-Passive</mode>
  <local-info><state>active</state><priority>100</priority><state-sync>Complete</state-sync></local-info>
  <peer-info><conn-status>up</conn-status><state>passive</state>
    <conn-ha1><conn-status>up</conn-status></conn-ha1><conn-ha2><conn-status>down</conn-status></conn-ha2>
  </peer-info>
  <running-sync>not synchronized</running-sync>
</group></result></response>
```

Fields read

Response field	How the tool uses it
`enabled, group/mode`	ha_enabled, ha_mode.
`local-info: state, priority, preemptive, state-sync`	This member's state and settings.
`peer-info: state, conn-status`	Peer state and the connection to it.
`peer-info/conn-ha1, conn-ha2`	HA1 (control) and HA2 (data) link status.
`running-sync`	Config sync between the pair.
`link-monitoring, path-monitoring`	Whether failover monitoring is enabled.

Findings raised

Severity	Finding raised when...
high	HA peer connection is not 'up'.
high	HA1 or HA2 link is not 'up'.
medium	HA config is not synchronised with the peer.

`session_info` Firewall session table

CLI equivalent: show session info

METHOD	REQUEST TYPE	REPORT SECTION	PERMISSION	CALLS PER RUN
POST /api/	**op**	**targeted**	**Operational Requests**	**1 per firewall**

How full the firewall's session table is, plus current connection and packet rates.

Parameters

Parameter	Value	Notes
`type`	`op`	Operational command.
`cmd`	`<show><session><info></info></session></show>`	Must match an OP_COMMANDS entry exactly, character for character.
`target`	`<firewall serial>`	Only for per-firewall checks. Letters, digits, - and _ only.

Example request

```
POST /api/ HTTP/1.1
Host: <panorama>
X-PAN-KEY: <XML API key>
Content-Type: application/x-www-form-urlencoded

type=op&
cmd=<show><session><info></info></session></show>&
target=007951000123456
```

Example response

Sample data from the test suite, trimmed.

```
<response status="success"><result>
  <num-max>100000</num-max><num-active>85000</num-active><cps>120</cps>
</result></response>
```

Fields read

Response field	How the tool uses it
`num-active, num-max`	session_util_pct = active / max (here 85.0 %).
`cps, pps, kbps`	Connections, packets and kilobits per second, for context.

Findings raised

Severity	Finding raised when...
high	Session table is 80 % full or more.

`dp_resources` Firewall dataplane load

CLI equivalent: show running resource-monitor minute last 1

METHOD	REQUEST TYPE	REPORT SECTION	PERMISSION	CALLS PER RUN
POST /api/	**op**	**targeted**	**Operational Requests**	**1 per firewall**

Dataplane (traffic processing) CPU and packet buffer use over the last minute. This is the best single indicator of whether a firewall is overloaded.

Parameters

Parameter	Value	Notes
`type`	`op`	Operational command.
`cmd`	`<show><running><resource-monitor><minute><last>1</last></minute></resource-monitor></running></show>`	Must match an OP_COMMANDS entry exactly, character for character.
`target`	`<firewall serial>`	Only for per-firewall checks. Letters, digits, - and _ only.

Example request

```
POST /api/ HTTP/1.1
Host: <panorama>
X-PAN-KEY: <XML API key>
Content-Type: application/x-www-form-urlencoded

type=op&
cmd=<show><running><resource-monitor><minute><last>1</last></minute></resource-monitor></running></show>&
target=007951000123456
```

Example response

Sample data from the test suite, trimmed.

```
<response status="success"><result><resource-monitor><data-processors><dp0><minute>
  <cpu-load-average>
    <entry><coreid>1</coreid><value>90</value></entry><entry><coreid>2</coreid><value>94</value></entry>
  </cpu-load-average>
  <resource-utilization><entry><name>packet buffer</name><value>12</value></entry></resource-utilization>
</minute></dp0></data-processors></resource-monitor></result></response>
```

Fields read

Response field	How the tool uses it
`cpu-load-average/entry/value`	Averaged over cores; the busiest dataplane is reported as dp_cpu_pct.
`resource-utilization: packet buffer`	packet_buffer_pct (highest value).

Findings raised

Severity	Finding raised when...
high	Dataplane CPU is 80 % or more.
high	Packet buffer use is 80 % or more.

`logging_status` Firewall log forwarding

CLI equivalent: show logging-status

METHOD	REQUEST TYPE	REPORT SECTION	PERMISSION	CALLS PER RUN
POST /api/	**op**	**targeted**	**Operational Requests**	**1 per firewall**

Shows whether the firewall is sending its logs to Panorama or a log collector, and how far behind it is. A firewall that stops forwarding logs leaves gaps in security monitoring.

Parameters

Parameter	Value	Notes
`type`	`op`	Operational command.
`cmd`	`<show><logging-status></logging-status></show>`	Must match an OP_COMMANDS entry exactly, character for character.
`target`	`<firewall serial>`	Only for per-firewall checks. Letters, digits, - and _ only.

Example request

```
POST /api/ HTTP/1.1
Host: <panorama>
X-PAN-KEY: <XML API key>
Content-Type: application/x-www-form-urlencoded

type=op&
cmd=<show><logging-status></logging-status></show>&
target=007951000123456
```

Example response

Sample data from the test suite, trimmed.

```
<response status="success"><result><![CDATA[
'Log Collection log forwarding agent' is active and connected to 192.0.2.50
   Type     Last Log Created      Last Log Fwded      Last Seq Num Fwded  Last Seq Num Acked
   config   2026/11/03 09:50:00   2026/11/03 09:50:00   100   100
   system   2026/11/03 09:55:00   2026/11/03 08:25:00   200   200
]]></result></response>
```

Fields read

Response field	How the tool uses it
`'is active and connected to <host>'`	log_forwarding = connected, log_destination = host.
`'not connected' / 'is not active'`	log_forwarding = not connected.
`Last Log Created vs Last Log Fwded`	log_fwd_lag_min: the largest gap across log types.

Findings raised

Severity	Finding raised when...
high	The firewall is not forwarding logs.
medium	Log forwarding is 60 minutes or more behind.

`system_info, system_resources, disk_space` (with target)

The same commands documented in section 3.1, sent to a firewall instead of Panorama.

METHOD	REQUEST TYPE	REPORT SECTION	PERMISSION	CALLS PER RUN
POST /api/	**op**	**targeted**	**Operational Requests**	**3 per firewall**

Adding target=<serial> makes Panorama run the command on that firewall. The request and response formats are the same as in section 3.1; what differs is how the result is used.

Parameters

Parameter	Value	Notes
`type`	`op`	Operational command.
`cmd`	`<one of the three commands in 3.1>`	Must match an OP_COMMANDS entry exactly, character for character.
`target`	`<firewall serial>`	Only for per-firewall checks. Letters, digits, - and _ only.

Example request

```
POST /api/ HTTP/1.1
Host: <panorama>
X-PAN-KEY: <XML API key>
Content-Type: application/x-www-form-urlencoded

type=op&
cmd=<show><system><resources></resources></system></show>&
target=007951000123456
```

Fields read

Response field	How the tool uses it
`system_info: av-release-date, app-release-date`	Age in days of the antivirus and apps & threats content.
`system_resources: %Cpu(s), Mem`	Firewall management CPU and memory, as for Panorama.
`disk_space: df -h lines`	Firewall disks, added to disk_usage.csv.

Findings raised

Severity	Finding raised when...
high	Management CPU 85 % or more, or memory 90 % or more.
high	A disk is 90 % full or more (medium at 80 %).
medium	Antivirus content is 3 or more days old.
medium	Applications & Threats content is 30 or more days old.
medium	Any per-firewall command failed (the error is in the report).

4. Health findings reference

After collection, evaluate() applies these rules. Each finding has a severity, an area, a target (the firewall or component) and a detail line. Findings are sorted by severity in the report and in findings.csv.

Severity	Area	Rule	Source call
critical	connectivity	Firewall not connected to Panorama	`devices_all`
critical	high-availability	HA pair has both members active (possible split brain)	`devices_all`
high	high-availability	HA state not healthy (e.g. suspended)	`devices_all`
high	high-availability	HA pair has no active member	`devices_all`
high	high-availability	HA peers run different PAN-OS or content versions	`devices_all`
high	high-availability	HA peer connection, HA1 or HA2 link not 'up'	`ha_all`
high	sync	Policy or template out of sync, or last commit/push failed	`devices_all` `devicegroups` `templates`
high	versions	Firewall runs a newer PAN-OS feature release than Panorama	`devices_all` `system_info`
high	panorama	Panorama HA peer down, or HA local state not active/passive	`ha_state`
high	panorama / resources	Management CPU >= 85 %, memory >= 90 %	`system_resources`
high	resources	Session table >= 80 %, dataplane CPU >= 80 %, packet buffer >= 80 %	`session_info` `dp_resources`
high	disk	A filesystem is >= 90 % full (log partition excluded)	`disk_space`
high	log-collector	Log collector not connected	`log_collectors`
high	jobs	A job failed, or a push failed on at least one firewall	`jobs_all`
high	logging	Firewall is not forwarding logs	`logging_status`
medium	high-availability	HA peer not connected to Panorama	`devices_all`
medium	high-availability	HA config not synchronised with the peer	`ha_all`
medium	panorama	Panorama HA running config not synchronised	`ha_state`
medium	panorama	Clock not synchronised to an NTP server	`ntp`
medium	disk	A filesystem is >= 80 % full	`disk_space`
medium	certificates	Device certificate missing/expired or expires within 30 days	`devices_all`
medium	versions	Version flagged as unsupported by Panorama	`devices_all`
medium	versions	Antivirus content 3+ releases behind the newest in use	`devices_all`
medium	versions	Antivirus content 3+ days old, or apps & threats 30+ days old	`system_info (target)`
medium	logging	Log forwarding 60+ minutes behind	`logging_status`
medium	system-logs	Critical system log events in the log window	`log get`
medium	resources	A per-firewall command failed	`any targeted call`
medium	collection	A section could not be collected	`any call`
low	versions	Firewall runs a newer maintenance release than Panorama	`devices_all` `system_info`
low	versions	Apps & threats or WildFire content behind the newest in use	`devices_all`
low	high-availability	HA peer not managed by Panorama, or no peer reported	`devices_all`
info	inventory	Firewall is deactivated in Panorama	`devices_all`
info	versions	PAN-OS version differs from the fleet majority	`devices_all`

Severity	Area	Rule	Source call
info	config-logs	Number of configuration changes in the log window	`log get`

5. Report outputs

Each run writes a new folder, `reports/<YYYYMMDD-HHMMSS>/`, containing:

File	Contents	Built from
`report.html`	Self-contained web page: summary tiles, findings and every table below, plus the list of API calls made	`all`
`findings.csv`	All health findings, most severe first	`evaluate()`
`panorama.csv`	Panorama system, HA, CPU/memory and NTP fields	`system_info, ha_state, system_resources, ntp`
`inventory.csv`	One row per managed firewall	`devices_all`
`firewall_status.csv`	One row per firewall: versions, sync, HA and logging together	`devices_all, sync, targeted`
`ha_pairs.csv`	HA pairs and their status	`devices_all`
`sync_status.csv`	Device-group and template membership with sync status	`devicegroups, templates`
`jobs.csv`	Last 50 Panorama jobs	`jobs_all`
`log_collectors.csv`	Log collectors and connection state	`log_collectors`
`system_logs.csv`	High/critical system log events, last 24 h	`log get`
`config_logs.csv`	Configuration changes, last 24 h	`log get`
`firewall_resources.csv`	Live per-firewall results (opt-in)	`targeted calls`
`disk_usage.csv`	Disk use for Panorama and checked firewalls	`disk_space`
`collection_errors.csv`	Any call that failed, with the reason and suggested fix	`any call`

6. Configuration reference (.env)

Settings are read from the .env file next to the script. The .env file is excluded from git. In the interactive menu, session settings can only make a run *lighter* than .env: the delay can only go up and the firewall limit can only go down.

Setting	Default	Meaning
PANORAMA_HOST	(required)	Panorama address. Must also be listed in ALLOWED_HOSTS.
ALLOWED_HOSTS	(required)	Hosts the tool may contact. Read from the .env file only; environment variables cannot change it.
PANORAMA_PORT	443	HTTPS port.
PANORAMA_XML_API_KEY	(required)	XML API key. PANORAMA_API_KEY is used if this is empty.
KEY_IN_HEADER	true	Send the key in the X-PAN-KEY header (never in a URL).
HTTP_METHOD	POST	POST keeps parameters out of the URL. GET is refused unless the key is in the header.
VERIFY_TLS	false	Check Panorama's HTTPS certificate. Lab default is off, with a warning.
CA_BUNDLE	(empty)	CA file to trust when VERIFY_TLS=true.
BYPASS_PROXY	true	Connect directly instead of through a system proxy.
REQUEST_DELAY_SEC	1	Minimum seconds between requests.
TIMEOUT_SEC	60	Seconds to wait for a reply.
LOG_WINDOW_HOURS	24	How far back the log searches go.
MAX_LOGS	100	Maximum entries per log search.
MAX_JOBS	50	Number of recent jobs kept in the report.
ENABLE_TARGETED	false	Run the per-firewall checks by default.
MAX_TARGETED_FIREWALLS	5	Maximum firewalls for per-firewall checks per run.
OUTPUT_DIR	reports	Where report folders are written.

Command-line options

Option	Effect
(no options)	Opens the interactive menu.
--dry-run	Prints every call a run would make. Sends nothing, writes no report.
--diagnose	Tests each permission and how the key is sent, then exits.
--sections a,b,c	Collect only these sections (default: panorama, devices, sync, jobs, collectors, logs, targeted).
--targeted	Switch on the per-firewall checks for connected firewalls (max 5).
--firewalls SERIAL,...	Per-firewall checks for these serials only.
--format html,csv	Which report files to write.
--output-dir DIR	Override OUTPUT_DIR.

7. Safety controls

The tool is built so that it **cannot** change state on Panorama or the firewalls. These controls are in the code and covered by automated tests, not just in configuration.

Control	How it works	Where in the code
Read-only allowlist	Only 14 exact <show> commands. Log searches only cover system and config logs.	`OP_COMMANDS` `guard()`
Checked before sending	Every request goes through guard() first. Anything else raises ReadOnlyViolation and the run stops before a byte is sent.	`PanoramaClient.request()`
No commits, pushes or changes	There is no code path for set, edit, delete, commit, push, import, clear, request, debug or test commands.	`guard()`
Lab host lock	Only hosts in ALLOWED_HOSTS in the local .env file. Environment variables cannot widen this.	`load_settings()`
Low load	One request at a time, at least 1 s apart, one network retry. Logs capped at 100 entries and 24 h. Per-firewall checks opt-in and capped at 5.	`_send()` `.env settings`
API key protection	The key travels in the X-PAN-KEY header, never in a URL, and is never written to reports or logs.	`request()`
Preview before running	--dry-run prints every call without sending anything.	`main()`
Automated tests	71 tests cover the guards, settings, client, parsers and full runs against a fake Panorama.	`tests/` `test_pano_healthcheck.py`

8. Python code structure

Everything is in one file, `pano_healthcheck.py`, in this order from top to bottom.

1. Settings	`load_settings(), Settings` Reads .env, applies the lab-host lock (ALLOWED_HOSTS), picks the API key
2. Guards	`OP_COMMANDS, guard()` The allowlist. Blocks anything that is not a read
3. API client	`PanoramaClient: request(), op(), log_query()` Sends requests one at a time; delay, single retry, error messages
4. XML helpers	`flatten(), top_entries()` Turns XML replies into simple {field: value} rows
5. Collectors	`Collector.collect_<section>()` One method per report section; a failure is recorded and the run continues
6. Parsers	`parse_ha(), parse_resources(), parse_disk_space(), ...` Read the text and XML of individual commands
7. Health findings	`evaluate()` Rules that turn data into critical / high / medium / low findings
8. Reports	`write_html(), write_csv(), csv_tables()` Self-contained report.html and CSV files
9. Interactive menu	`Tui, MenuApp` Arrow-key menu; uses the same client and collectors as the CLI
10. Command line	`main(), run_health_check()` --dry-run, --diagnose, --sections, --targeted, --format

How a call is made in code

```
# Collector (section "devices") asks the client by name, never with free-form XML:
result = self.client.op("devices_all")

# PanoramaClient.op() looks the exact XML up in the allowlist ...
params = {"type": "op", "cmd": OP_COMMANDS["devices_all"]}

# ... and request() checks it before anything is sent:
guard(params)            # raises ReadOnlyViolation if not allowed
status, payload = self._send("/api/", urlencode(params), headers, "POST")
```

The allowlist (shortened)

```
OP_COMMANDS = {
    "system_info":  "<show><system><info></info></system></show>",
    "devices_all":  "<show><devices><all></all></devices></show>",
    "jobs_all":     "<show><jobs><all></all></jobs></show>",
    ...                                  # 14 commands in total, all <show>
}
```

Appendix A. Command allowlist

The complete list of operational commands the tool can send. Commands marked as forwardable may be sent to a firewall with target=serial; the others only go to Panorama.

Name	Exact XML sent as cmd	Forwardable
system_info	<show><system><info></info></system></show>	yes
ha_state	<show><high-availability><state></state></high-availability></show>	no
devices_all	<show><devices><all></all></devices></show>	no
devicegroups	<show><devicegroups></devicegroups></show>	no
templates	<show><templates></templates></show>	no
jobs_all	<show><jobs><all></all></jobs></show>	no
log_collectors	<show><log-collector><connected></connected></log-collector></show>	no
session_info	<show><session><info></info></session></show>	yes
system_resources	<show><system><resources></resources></system></show>	yes
disk_space	<show><system><disk-space></disk-space></system></show>	yes
ntp	<show><ntp></ntp></show>	yes
ha_all	<show><high-availability><all></all></high-availability></show>	yes
logging_status	<show><logging-status></logging-status></show>	yes
dp_resources	<show><running><resource-monitor><minute><last>1</last></minute></resource-monitor></running></show>	yes

ntp is forwardable but is only used against Panorama in the current health check.